M21 Methodical Debugging

Methodology Intermediate

Structured approach to diagnosing and resolving errors

The blind iteration problem

The temptation is strong to describe a bug vaguely and let Claude try fixes in series. This pattern is costly in tokens and time, and often produces "fixes" that mask the problem without resolving it. A structured three-phase approach radically changes debugging effectiveness.

Phase 1: Reproduce

Before analyzing or fixing anything, obtain a reliable reproduction. A bug that cannot be reproduced deterministically is a bug that cannot be fixed reliably.

Give Claude the minimum necessary to reproduce:

- Exact error message (copy-paste, not a summary)
- Full stack trace if available
- Command or sequence of actions that trigger the error
- Environment: OS, Node/Python/etc. version, relevant env variables

Phase 2: Isolate

Create a minimal test case that reproduces the bug. Isolation serves two purposes: it confirms you have correctly understood the cause, and it immediately provides a regression test once the bug is fixed.

```
# Example: isolating a parsing bug
node -e "
const parse = require('./src/parser');
const input = 'problematic case';
console.log(parse(input));
"
```

Ask Claude to explain its hypothesis before each change. If Claude cannot articulate why it is modifying a file, that is a signal of blind iteration.

Phase 3: Fix and verify

Once the cause is isolated, apply the minimal fix. Avoid using the debug context to refactor adjacent code: scope creep complicates validation.

Two-step verification: the minimal test case passes, then the full test suite passes. If an existing test breaks after the fix, the fix has unanticipated side effects.

Structured debug prompt

```
Bug: [one-sentence description]
Exact error: [message + stack trace]
Reproduction: [command / steps]
Already tried: [previous attempts]

Before modifying anything,
list your hypotheses in order of probability.
```

This prompt forces Claude to expose its reasoning, allowing you to validate or correct the direction before it writes a single line of code.

When to use debug agents

A single agent suits bugs localized to 1 to 3 files. For problems that cross multiple interconnected components (a network bug affecting auth affecting the database), delegating exploration to a sub-agent via the Agent tool (formerly Task) preserves the main context.

```
Main agent → Agent: explore auth flow from request to DB
             └─ Sub-agent explores and returns a summary
Main agent receives the summary (not the raw context)
```

A sub-agent returns a smaller summary, but errors in that summary can still influence the main agent. Verify consequential findings.

Warning signals during debugging

Signal	Interpretation
Claude modifies a file without explanation	Ask "why this file?"
Fix that works around the error	Look for root cause, not symptom
Tests that pass but behavior unchanged	Test not adapted to the real bug
Same error after fix	Reproduction insufficiently isolated

Resetting the context

Long or irrelevant context can make important constraints harder to apply; there is no fixed turn count that predicts this. If debugging drags on, use `/compact` to summarize the history or `/clear` to restart with a clean context containing only the essential elements.